

Trabajo práctico Nro 1

Filsinger Gonzalo, Perea Ignacio, Parfait Manuel

*Electrónica Aplicada II 4R2, Ingeniería Electrónica, Facultad Regional Córdoba
Universidad Tecnológica Nacional*

Resumen—En este informe se abordará el desarrollo del trabajo práctico Nro 2, en el cual experimentaremos con las interrupciones externas del microcontrolador STM32F103C8T6, utilizando la placa BluePill. Se desarrollaran los primeros dos ejercicios del trabajo práctico.

Index Terms—BluePill, GPIO, LED, Memoria, Interrupciones, Botones.

I. INTRODUCCIÓN

EN este informe solamente se abordarán los primeros dos ejercicios del trabajo práctico, en el primero utilizaremos dos botones uno con mas prioridad que el otro para interrumpir el parpadeo de un led, y en el segundo ejercicio haremos uso del ADC para medir la tensión de salida de un potenciómetro (Hasta 3.3V), y temperatura de un sensor interno, según un boton que cambia entre la lectura del potenciómetro y la lectura del sensor de temperatura.

II. EJERCICIO 1: INTERRUPTIONES CON BOTONES DE BAJA Y ALTA PRIORIDAD

Para este programa se usará el código del TP1 como base, el cual se modificará para hacer interrupciones al led del TP1 con botones de baja y alta prioridad. Para conseguir esto se agregaron las siguientes definiciones en el archivo main.c:

- Se definió la dirección base **AFIO_BASE**
- Se definió la dirección base **EXTI_BASE**
- Se definió la dirección base **NVIC_BASE**
- Se definió el registro **AFIO_EXTICR1**
- Se definió el registro **EXTI_IMR**
- Se definió el registro **EXTI_FTSR**
- Se definió el registro **EXTI_PR**
- Se definió el registro **NVIC_ISER0**
- Se definió el registro **NVIC_IPR**
- Se definieron las mascaras **GPIOA6** y **GPIOA7**

```
#include <stdint.h>

// --- Direcciones Base ---
#define RCC_BASE    0x40021000
#define GPIOC_BASE  0x40011000
#define GPIOA_BASE  0x40010800
#define AFIO_BASE   0x40010000
#define EXTI_BASE   0x40010400
#define NVIC_BASE   0xE000E100

// --- Registros de Periféricos ---
#define RCC_APB2ENR  *(volatile uint32_t *) (RCC_BASE + 0x18)
#define GPIOC_CRH    *(volatile uint32_t *) (GPIOC_BASE + 0x04)
#define GPIOA_CRL    *(volatile uint32_t *) (GPIOA_BASE + 0x00)
#define GPIOC_ODR    *(volatile uint32_t *) (GPIOC_BASE + 0x0C)
#define GPIOA_ODR    *(volatile uint32_t *) (GPIOA_BASE + 0x0C)

#define AFIO_EXTICR1 *(volatile uint32_t *) (AFIO_BASE + 0x08)
#define EXTI_IMR     *(volatile uint32_t *) (EXTI_BASE + 0x00)
#define EXTI_FTSR    *(volatile uint32_t *) (EXTI_BASE + 0x0C)
#define EXTI_PR      *(volatile uint32_t *) (EXTI_BASE + 0x14)
#define NVIC_ISER0   *(volatile uint32_t *) (NVIC_BASE + 0x00)
#define NVIC_IPR     ((volatile uint8_t *) (NVIC_BASE + 0x400))

// --- Campos de Bits ---
#define RCC_AFIOEN   (1 << 0)
#define RCC_IOPAEN   (1 << 2)
#define RCC_IOPCEN   (1 << 4)
#define GPIOC13      (1UL << 13)
#define GPIOA7       (1UL << 7) // LED Normalidad
#define GPIOA6       (1UL << 6) // LED Botón Baja Prioridad
#define GPIOA5       (1UL << 5) // LED Botón Alta Prioridad
```

Definiciones de los registros en main.c

Aquí se configuran los registros necesarios para poder realizar las interrupciones con los botones. El registro **AFIO_EXTICR1** se utiliza para configurar el pin del botón como fuente de interrupción externa; en este caso usaremos dos, el A1 y el A2. El registro **EXTI_IMR** se utiliza para habilitar las interrupciones de los pines configurados. El registro **EXTI_FTSR** se utiliza para configurar la interrupción por flanco de bajada. El registro **NVIC_ISER0** se utiliza para habilitar las interrupciones en el NVIC, y el registro **NVIC_IPR** se utiliza para configurar la prioridad de las interrupciones.

A continuación agregamos las funciones de interrupción para los botones, las cuales se encargan de encender y apagar el led correspondiente al botón presionado. Para el botón de

baja prioridad (A1) se encenderá el led del pin A6, y para el botón de alta prioridad (A2) se encenderá el led del pin A7.

```
// Botón en PA1 (EXTI1) - PRIORIDAD BAJA -> Titila PA6 5 veces
void EXTI1_IRQHandler(void)
{
    if (EXTI_PR & (1 << 1))
    {
        // Limpiamos el flag primero para que futuras pulsaciones se registren correctamente
        EXTI_PR |= (1 << 1);

        // Encendemos y apagamos 5 veces
        for (int i = 0; i < 10; i++)
        {
            GPIOA_ODR ^= GPIOA6;
            delay_ms_bloqueante(200);
        }

        GPIOA_ODR &= ~GPIOA6;
    }
}
```

Agregado de funciones para interrupciones

```
// Botón en PA2 (EXTI2) - PRIORIDAD ALTA -> Titila PA5 5 veces
void EXTI2_IRQHandler(void)
{
    if (EXTI_PR & (1 << 2))
    {
        // Limpiamos el flag
        EXTI_PR |= (1 << 2);

        // Encendemos y apagamos 5 veces
        for (int i = 0; i < 10; i++)
        {
            GPIOA_ODR ^= GPIOA5;
            delay_ms_bloqueante(200);
        }

        GPIOA_ODR &= ~GPIOA5;
    }
}
```

Agregado de funciones para interrupciones

Ahora pasamos a modificar la función **main**

```
void main(void)
{
    // 1. RCC (Relojes)
    RCC_APB2ENR |= RCC_AFIOEN | RCC_IOPAEN | RCC_IOPCEN;

    // 2. GPIO
    // PC13 (LED placa) como salida
    GPIOC_CRH &= 0xFF0FFFFF;
    GPIOC_CRH |= 0x00200000;

    // Configurar PA7, PA6 y PA5 como SALIDAS push-pull (2)
    // Configurar PA2 y PA1 como ENTRADAS pull-up/down (8)
    GPIOA_CRL &= 0x000FF00F;
    GPIOA_CRL |= 0x22200880;

    // Activamos resistencias Pull-up internas en PA1 y PA2
    GPIOA_ODR |= (1 << 1) | (1 << 2);

    // 3. AFIO (Multiplexor)
    AFIO_EXTICR1 &= ~(0x00000FF0);

    // 4. EXTI
    EXTI_IMR |= (1 << 1) | (1 << 2);
    EXTI_FTSR |= (1 << 1) | (1 << 2);

    // 5. NVIC PRIORIDADES (Ajustadas para permitir que SysTick no se congele)
    NVIC_IPR[7] = 0x20; // Prioridad BAJA para Botón PA1 (IRQ 7)
    NVIC_IPR[8] = 0x10; // Prioridad ALTA para Botón PA2 (IRQ 8)
    // Nota: SysTick mantiene su prioridad por defecto que es 0x00 (Máxima)

    // 6. NVIC ENABLE
    NVIC_ISER0 |= (1 << 7) | (1 << 8);

    systick_init_ms();

    while (1)
    {
        // --- Tarea de Normalidad (Background Task) ---
        GPIOA_ODR ^= GPIOA7;
        GPIOC_ODR ^= GPIOC13;
        delay_ms_bloqueante(500);
    }
}
```

Configuración de los pines y las interrupciones en main

Primero habilitamos los clocks del GPIO y AFIO, luego configuramos los pines A5, A6 y A7 como salida, y los A1 y A2 como entrada con pull-up. Luego asociamos el pin GPIO a la línea EXTI y pasamos a configurar las interrupciones como flancos descendentes, por ultimo configuramos el NVIC con las prioridades correspondientes.

II-A. Imagenes del Funcionamiento

III. EJERCICIO 2: USO DE ADC

Ahora se busca controlar mediante el ADC unos leds que registran el voltaje de salida de un potenciómetro, o la opción 2, leer la temperatura del sensor interno del microcontrolador, como el ADC se configura en el PA1, usaremos el PA2 para el botón. Para esto se agregaron las siguientes definiciones en el archivo main.c:

```
// --- Registros del ADC1 ---
#define ADC1_BASE 0x40012400
#define ADC1_SR *(volatile uint32_t *) (ADC1_BASE + 0x00)
#define ADC1_CR2 *(volatile uint32_t *) (ADC1_BASE + 0x08)
#define ADC1_SMPR1 *(volatile uint32_t *) (ADC1_BASE + 0x0C)
#define ADC1_SMPR2 *(volatile uint32_t *) (ADC1_BASE + 0x10)
#define ADC1_SQR3 *(volatile uint32_t *) (ADC1_BASE + 0x34)
#define ADC1_DR *(volatile uint32_t *) (ADC1_BASE + 0x4C)

// --- Campos de Bits ---
#define RCC_AFI0EN (1 << 0)
#define RCC_IOPAEN (1 << 2)
#define RCC_IOPCEN (1 << 4)
#define RCC_ADC1EN (1 << 9)

#define GPIOC13 (1UL << 13)
#define GPIOA4 (1UL << 4) // Nuevo LED 1
#define GPIOA5 (1UL << 5) // Nuevo LED 2
#define GPIOA6 (1UL << 6) // Nuevo LED 3
#define GPIOA7 (1UL << 7) // Nuevo LED 4

#define ADC_CR2_ADON (1 << 0)
#define ADC_CR2_SWSTART (1 << 22)
#define ADC_CR2_TSVREFE (1 << 23)
#define ADC_SR_EOC (1 << 1)
```

Definiciones de los registros para interrupciones

- Se definió el registro **ADC1_BASE** con la dirección base del ADC, el cual se utiliza para configurar y controlar el funcionamiento del ADC.
- Se definió el registro **ADC1_SR** el cual se utiliza para verificar el estado del ADC, como por ejemplo si la conversión ha terminado o si hay algún error.
- Se definió el registro **ADC_CR2** el cual se utiliza para configurar el ADC, como por ejemplo habilitar el ADC, configurar el modo de conversión, etc.
- Se definió el registro **ADC_SMPR1** el cual se utiliza para configurar el tiempo de muestreo del ADC, lo cual es importante para obtener una conversión precisa.
- Se definió el registro **ADC_SMPR2** el cual se utiliza para configurar el tiempo de muestreo del ADC para los canales 0 a 9.
- Se definió el registro **ADC_SQR3** el cual se utiliza para configurar el canal que se va a convertir.
- Se definió el registro **ADC_DR** el cual se utiliza para leer el resultado de la conversión del ADC.

Luego se agregaron las siguientes funciones para configurar el SysTick y generar el retardo por interrupciones:

```
// Botón en PA2 (EXTI2) - PRIORIDAD ALTA -> Titila PA5 5 veces
void EXTI2_IRQHandler(void)
{
    if (EXTI_PR & (1 << 2))
    {
        // Limpiamos el flag
        EXTI_PR |= (1 << 2);

        // Encendemos y apagamos 5 veces
        for (int i = 0; i < 10; i++)
        {
            GPIOA_ODR ^= GPIOA5;
            delay_ms_bloqueante(200);
        }

        GPIOA_ODR &= ~GPIOA5;
    }
}
```

Funciones para configurar el SysTick y generar el retardo por interrupciones

- La función **SysTick_Handler** es la rutina de servicio de interrupción del SysTick, esta función se ejecuta cada vez que se genera una interrupción, en esta función se incrementa la variable global **tick** para contar la cantidad de interrupciones generadas.
- La función **sysTick_init_ms** es la función que se utiliza para configurar el SysTick para generar interrupciones cada cierto tiempo. Esta pone en 0 la variable **tick** cada vez que se inicia, hace que se divida el reloj del procesador en 8, se configura el disparo en *1ms*, inicializa la variable **VAL** en 0 y habilita el contador y las interrupciones del SysTick.
- La función **delay_ms_bloqueante** es la función que se utiliza para hacer parpadear los leds utilizando el retardo por interrupciones.

Con esto en cuenta lo que se modificó en la función **main** para generar el retardo por interrupciones fue lo siguiente:

```

systick_init_ms();
while (1)
{

    GPIOC_ODR &= ~GPIOC13;
    delay_ms_bloqueante(500);
    GPIOC_ODR |= GPIOC13;
    delay_ms_bloqueante(500);

    GPIOA_ODR |= GPIOA7;
    delay_ms_bloqueante(500);
    GPIOA_ODR &= ~GPIOA7;
    delay_ms_bloqueante(500);
}

```

Función main con retardo por interrupciones

IV. CONCLUSIÓN

Se logró poner en funcionamiento la BluePill, y se logró hacer andar un led usando sus direcciones GPIO. Para esto se tuvo que usar los comandos para la placa clon, además de modificar el Makefile para añadir dicho comando y así que funcione de manera correcta. Lo más difícil de este trabajo fue comenzar a pensar el código en base a los registros, ya que es un nivel de abstracción al que no estábamos acostumbrados, pero una vez que logramos comprender el funcionamiento del código logramos comprender mejor como funciona un microcontrolador a nivel de hardware, nos resultó particularmente interesante como un programa de alto nivel pasa a ser encender o apagar determinados bits de ciertos registros.

V. CODIGOS

V-A. main.c

```

1 #include <stdint.h>
2 // register address
3 #define RCC_BASE 0x40021000
4 #define GPIOC_BASE 0x40011000
5 #define GPIOA_BASE 0x40010800
6 #define RCC_APB2ENR *(volatile uint32_t *) (RCC_BASE
7 + 0x18)
8 #define GPIOC_CRH *(volatile uint32_t *) (GPIOC_BASE
9 + 0x04)
10 #define GPIOA_CRL *(volatile uint32_t *) (GPIOA_BASE
11 + 0x00)
12 #define GPIOC_ODR *(volatile uint32_t *) (GPIOC_BASE
13 + 0x0C)
14 #define GPIOA_ODR *(volatile uint32_t *) (GPIOA_BASE
15 + 0x0C)
16 // bit fields
17 #define RCC_IOPCEN (1 << 4)
18 #define RCC_IOPAEN (1 << 2)
19 #define GPIOC13 (1UL << 13)

```

```

15 #define GPIOA7 (1UL << 7)
16 // --- Registros del SysTick (Cortex-M Core) ---
17 #define SysTick_BASE 0xE000E010
18 #define SysTick_CTRL *(volatile uint32_t *) (
19 SysTick_BASE + 0x00)
20 #define SysTick_LOAD *(volatile uint32_t *) (
21 SysTick_BASE + 0x04)
22 #define SysTick_VAL *(volatile uint32_t *) (
23 SysTick_BASE + 0x08)
24 // Bits de control de SysTick
25 #define SysTick_CTRL_ENABLE (1 << 0) // Habilita
26 el contador
27 #define SysTick_CTRL_TICKINT (1 << 1) // Habilita
28 la interrupción de SysTick
29 #define SysTick_CTRL_CLKSOURCE (1 << 2) // Fuente de
30 reloj
31 #define SysTick_CTRL_COUNTFLAG (1 << 16) // Bandera
32 de desborde
33 volatile uint32_t tick;
34 // función que atiende la interrupción
35 void SysTick_Handler(void)
36 {
37     tick++;
38 }
39 // inicialización del systick
40 void systick_init_ms(void)
41 {
42     tick = 0;
43     SysTick_CTRL &= ~SysTick_CTRL_CLKSOURCE; //
44     Reloj de SysTick a HCLK/8 (8MHz / 8 = 1MHz -> 1
45     tick = 1us)
46     SysTick_LOAD = 999; // Poner valor de RECARGA
47     para 1000 ticks (1ms) (1000 - 1) = 999
48     SysTick_VAL = 0; // reset del contador
49     SysTick_CTRL |= SysTick_CTRL_TICKINT |
50     SysTick_CTRL_ENABLE; // Habilitar la interrupció
51     n de SysTick (TICKINT) Y el contador (ENABLE)
52 }
53 // función delay
54 void delay_ms_bloqueante(uint32_t ms)
55 {
56     uint32_t tiempo_inicio = tick + ms;
57     // Espera activa hasta que el contador global
58     haya avanzado 'ms' milisegundos
59     while (tick != tiempo_inicio);
60 }
61 void main(void)
62 {
63     RCC_APB2ENR |= RCC_IOPCEN;
64     RCC_APB2ENR |= RCC_IOPAEN;
65     GPIOC_CRH &= 0xFF0FFFFFFF;
66     GPIOC_CRH |= 0x00200000;
67     GPIOA_CRL &= 0x0FFFFFFF;
68     GPIOA_CRL |= 0x20000000;
69     systick_init_ms();
70     while (1)
71     {
72         GPIOC_ODR &= ~GPIOC13;
73         delay_ms_bloqueante(500);
74         GPIOC_ODR |= GPIOC13;
75         delay_ms_bloqueante(500);

76         GPIOA_ODR |= GPIOA7;
77         delay_ms_bloqueante(500);
78         GPIOA_ODR &= ~GPIOA7;
79         delay_ms_bloqueante(500);
80     }
81 }

```

Listing 1. Configuración inicial en main.c

V-B. crt.s

```

1 .syntax unified
2 .cpu cortex-m3
3 .thumb
4 .extern _esstack
5
6 .section .text.manejadores // definimos la seccion
   text.manejadores
7
8 // función básica para manejar los punteros
9 .thumb_func
10 .weak Default_Handler
11 Default_Handler:
12     b . // Bucle infinito */
13
14 // definir los vectores ejemplo
15 .weak NMI_Handler
16 .thumb_set NMI_Handler, Default_Handler
17 .weak SysTick_Handler
18 .thumb_set SysTick_Handler, Default_Handler
19 .weak HardFault_Handler
20 .thumb_set HardFault_Handler, Default_Handler
21
22 // vectores
23 .section .isr_vector,"a",%progbits
24 .word _esstack
25 .word _reset
26 .word NMI_Handler // 2: NMI (Non-Maskable
   Interrupt) */
27 .word HardFault_Handler // 3: HardFault (falla
   grave) */
28 .word 0 // 4: MemManage (falla de
   memoria) */
29 .word 0 // 5: BusFault (falla de
   bus) */
30 .word 0 // 6: UsageFault (falla
   de uso) */
31 .word 0 // 7: Reservado */
32 .word 0 // 8: Reservado */
33 .word 0 // 9: Reservado */
34 .word 0 // 10: Reservado */
35 .word 0 // 11: SVCALL (Llamada al
   Supervisor) */
36 .word 0 // 12: Debug Monitor */
37 .word 0 // 13: Reservado */
38 .word 0 // 14: PendSV (Interrupció
   n pendiente del sistema) */
39 .word SysTick_Handler // 15: El handler del
   SysTick */
40
41 // declaramos la función _reset dentro de la seccion
   text.reset
42 .section .text.reset
43 .thumb_func
44 .global _reset
45 _reset:
46     bl main
47     b .

```

V-C. linker.ld

```

1 MEMORY
2 MEMORY
3 {
4     FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 64K
5     RAM (xrw) : ORIGIN = 0x20000000, LENGTH = 20K
6 }
7
8 /* 2. Definición de Símbolos Globales */
9 ENTRY(_reset) /* El punto de entrada, definido en tu
   crt.s */
10
11 /* Símbolo para el final de la pila, usado en tu .
   isr_vector */
12 _esstack = ORIGIN(RAM) + LENGTH(RAM); /* 0x20005000
   (el final de los 20K de RAM) */

```

```

13 /* 3. Definición de las Secciones de Salida */
14 SECTIONS
15 {
16     /* SECCIONES EN FLASH */
17     /* La tabla de vectores debe ir primero en la
   Flash */
18     .isr_vector :
19     {
20         KEEP(*(.isr_vector)) /* Mantiene la sección .
   isr_vector de tu crt.s */
21     } > FLASH
22
23     /* Código del programa y datos de solo lectura */
24     .text :
25     {
26         *(.text) /* Todo el código C y
   ensamblador */
27         *(.text*) /* Variantes de .text */
28         *(.rodata) /* Datos de solo lectura (
   constantes) */
29         *(.rodata*)
30     } > FLASH
31
32     /* SECCIONES EN RAM */
33     /* .data: Variables inicializadas con un valor
   distinto de cero (ej: int x = 10;) */
34     /* El linker necesita símbolos para que el startup
   mueva los datos */
35     _sidata = LOADADDR(.data); /* Dirección de los
   valores en FLASH */
36
37     .data :
38     {
39         _sidata = .; /* Inicio de .data en RAM
   */
40         *(.data)
41         *(.data*)
42         _edata = .; /* Fin de .data en RAM */
43     } > RAM AT> FLASH /* Ubicada en RAM, pero
   cargada desde FLASH */
44     /* .bss: Variables inicializadas a cero o sin
   inicializar */
45     /* (ej: static int y; o int z = 0;) */
46     .bss :
47     {
48         _sbss = .; /* Símbolo de inicio de .
   bss */
49         *(.bss)
50         *(.bss*)
51         *(COMMON)
52         _ebss = .; /* Símbolo de fin de .bss
   */
53     } > RAM /* Solo ocupa espacio en RAM
   */
54 }

```

V-D. makefile

```

1 SRC_DIR = src
2 BUILD_DIR = build
3 CC = arm-none-eabi-gcc
4 AS = arm-none-eabi-as
5 LD = arm-none-eabi-ld
6 BIN = arm-none-eabi-objcopy
7 CFLAGS = -mthumb -mcpu=cortex-m3 -o0 -g
8
9 all: app.bin
10
11 $(BUILD_DIR):
12     mkdir $(BUILD_DIR)
13
14 crt.o: $(BUILD_DIR) $(SRC_DIR)/crt.s
15     $(AS) -o $(BUILD_DIR)/crt.o $(SRC_DIR)/crt.s
16
17 main.o: $(BUILD_DIR) $(SRC_DIR)/main.c

```

```
18     $(CC) $(CFLAGS) -c -o $(BUILD_DIR)/main.o $(
19     SRC_DIR)/main.c
20 app.elf: $(BUILD_DIR) $(SRC_DIR)/linker.ld crt.o
21     main.o
22     $(LD) -T $(SRC_DIR)/linker.ld -o $(BUILD_DIR)/
23     app.elf $(BUILD_DIR)/crt.o $(BUILD_DIR)/main.o
24
25 app.bin: app.elf
26     $(BIN) -O binary $(BUILD_DIR)/app.elf $(
27     BUILD_DIR)/app.bin
28
29 clean:
30     rm -rf $(BUILD_DIR)
31
32 openocd-flash: app.bin
33     openocd -f interface/stlink.cfg -c "set CPUTAPID
34     0x2ba01477" -f target/stm32flx.cfg \
35     -c "init" -c "reset halt" \
36     -c "flash write_image erase $(BUILD_DIR)/app.bin
37     0x08000000" \
38     -c "reset run" -c "exit"
39
40 openocd-erase:
41     openocd -f interface/stlink.cfg -c "set CPUTAPID
42     0x2ba01477" -f target/stm32flx.cfg \
43     -c "init" -c "reset halt" \
44     -c "flash erase_sector 0 0 last" -c "reset run"
45     -c "exit"
```